1. Docker Auto-Restart Behavior — Ensuring Containers Survive Reboots

🔍 **What is it?**

When running containers in production or on a server, we want them to automatically restart:

- If the container crashes due to an error
- If the Docker daemon restarts
- If the system reboots

⚙️ **How to achieve this?**

Docker provides a --restart policy flag for docker run. This flag tells Docker **how to behave when a container exits**.

🚦 **Restart Policy Types**

Policy	Description
no (default)	Docker does not restart the container automatically.
on-failure[:max]	Restarts container only if it exits with a non-zero code. You can limit the number of restarts.
always	Always restarts the container if it stops, regardless of exit status.
unless-stopped	Same as always, **but won't restart if the container was manually stopped**. Survives daemon and system reboots. Most commonly used in production.

🧪 **Example Command**

bash

CopyEdit

```
docker run -d --restart unless-stopped myapp-image
```

🛡️ **Best Practice**

- Use unless-stopped for services that should **persist across reboots** but can be manually disabled.
- Use on-failure for tasks that should not endlessly restart if they are failing due to logic errors.

✅ 2. AWS S3 — Public vs. Private Objects — Security Model

📄 What is S3?

Amazon S3 (Simple Storage Service) is an object storage service used for storing files like logs, backups, website assets, and data lakes.

🔒 Default Security Model

- **Buckets and objects are private by default.**
- Only the bucket/object owner (the AWS account that created them) has access unless explicitly shared.

🌐 Making S3 Objects Public

Ways to make data public:

1. **Bucket Policy** (recommended way for public access)
2. **Object-level ACLs** (legacy method — now discouraged)
3. **Turning off "Block Public Access"** setting

🔐 AWS S3 Security Features

e ɔtion

Policiesbased permissions at bucket-level. Can allow/deny access to all or specif

ɔlicies l S3 access at the **user or role** level.

:k Publicsetting to prevent accidental public access to buckets and objects.

tion **E-S3** or **SSE-KMS** to encrypt data at rest.

g and A S3 Access Logs and **CloudTrail** to monitor usage.

🚫 Risks of Public Access

- Unintended data leaks (common reason for security breaches)
- GDPR or compliance violations
- Open access to malicious users

✅ 3. AWS S3 — Static Website Hosting

🌐 What Is It?

S3 can host a **static website** directly — no backend, just HTML, CSS, JavaScript.

📂 Typical Use Cases

- Portfolio websites
- Landing pages
- Error pages or maintenance pages
- Hosting frontend of SPAs (with CloudFront)

🧭 Steps to Set Up Static Website Hosting

1. **Create an S3 Bucket**
   - Name it **exactly** as your domain name (e.g., example.com).
   - Disable **Block Public Access** setting.
2. **Upload Site Files**
   - Typically include index.html, error.html, CSS, images.
3. **Enable Static Website Hosting**
   - Go to bucket properties → Enable static hosting.
   - Set index.html and error.html.
4. **Add Bucket Policy to Make Files Public**

json

CopyEdit

```
{
  "Version":"2012-10-17",
  "Statement":[{
    "Effect":"Allow",
    "Principal": "*",
    "Action":"s3:GetObject",
    "Resource":"arn:aws:s3:::your-bucket-name/*"
  }]
}
```

5. **Access Your Website**
    - Via the website endpoint:

arduino

CopyEdit

http://your-bucket-name.s3-website-<region>.amazonaws.com

6. **(Optional) Add Custom Domain + SSL**
    - Use **Route 53 + CloudFront** to:
        - Serve from your domain (www.example.com)
        - Add free HTTPS via ACM (SSL)

✅ 4. Auto-Scaling Debugging [Extension]

🤖 What Is Auto Scaling?

AWS Auto Scaling helps you:

- Add instances when load is high
- Remove instances when load drops
- Maintain high availability and cost-efficiency

Auto Scaling can be based on:

- **Metrics (e.g., CPU usage)**
- **Scheduled scaling**
- **Step scaling (e.g., add 2 instances if CPU > 80%)**

🛠️ Common Auto Scaling Issues & Debugging

Issue	Cause	Solution
❌ Scaling doesn't happe	Alarms not triggering	Check if metric crosses threshold, verify alarm state in CloudWatch
🧯 Too frequent scaling	No cooldown or aggressive thresholds	Set **cooldown period**, use **step scaling**
🏗️ EC2 launch failures	AMI not found, wrong subnet, missing IAM role	View Activity History, fix launch template
🚑 Health check failures	Instance starts but is marked unhealthy	Review ELB health check config (ports, response time), check SG/NACL

🧪 Troubleshooting Tools

- **CloudWatch Logs**: Check if alarms are triggering as expected.
- **Auto Scaling Activity History**: See what actions were taken and why.
- **Elastic Load Balancer Logs**: Monitor health check failures.
- **CloudTrail**: Review who made changes to policies or scaling configs.
- **EC2 Logs**: Check /var/log/cloud-init.log, /var/log/messages for instance boot issues.

🛡️ Best Practices for Auto Scaling

- Use **target tracking** for simple metric-based scaling.
- For critical apps, configure **min** and **max** capacity properly.
- Use **multi-AZ** placement for high availability.
- Combine with **CloudWatch alarms** and **SNS notifications** to stay informed.

✅ What is GitHub Version Control?

Version Control using Git and GitHub allows you to:

- Track code changes over time
- Collaborate without overwriting each other's work
- Revert back to older versions if needed
- Work on multiple features/bugs in parallel using **branches**

⚙️ Setting Up Git and GitHub

✅ Initial Setup:

bash

CopyEdit

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

✅ Clone a Repository:

bash

CopyEdit

```
git clone https://github.com/username/repo-name.git
cd repo-name
```

🌿 Branching — Work Independently

Branch = A parallel line of development from the main code (usually main or master).

✅ Create and Switch to a New Branch

bash

CopyEdit

```
git checkout -b feature-login
```

Now you're working in an isolated branch, safe from affecting the main code.

💾 Stage, Commit, and Push Changes

✅ **After modifying files:**

bash

CopyEdit

```
git add .
git commit -m "Added login form"
```

✅ **Push Your Branch to GitHub**

bash

CopyEdit

```
git push origin feature-login
```

🔁 Pull Request (PR) — Collaborate & Review

A **pull request** is asking to merge your changes from a branch (like feature-login) into another branch (like main), usually after peer review.

✅ **Steps to Create a Pull Request:**

1. Go to your GitHub repository.
2. Click **“Compare & Pull Request”** (after pushing your branch).
3. Add a title and description of your changes.
4. Click **“Create Pull Request”**.

🧠 **Best Practices:**

- Clearly describe what your PR does.
- Assign reviewers.
- Link issues (e.g., “Closes #42”).

🔀 Merging Branches — Integrate Changes

Once PR is reviewed and approved:

✅ **Merge via GitHub UI:**

Click **“Merge Pull Request”** on GitHub, then delete the branch if no longer needed.

✅ **Or merge locally:**

```
git checkout main
git pull origin main
git merge feature-login
git push origin main
```

⚠️ Conflict Resolution — What if Files Clash?

✅ **If files are changed in both branches:**

Git will alert you of a **merge conflict**.

You must manually resolve it by editing the conflicting files. Git marks the conflict like this:

```
<<<<<<< HEAD
your current branch content
=======
incoming branch content
>>>>>>> feature-login
```

✅ **After fixing conflicts:**

```
git add .
git commit -m "Resolved merge conflict"
git push
```

🔄 Syncing with Remote Changes

While working, others might push to main. To keep your branch up-to-date:

✅ **Rebase your branch:**

```
git fetch origin
git rebase origin/main
```

OR

✅ **Merge main into your branch:**

bash

CopyEdit

```
git checkout feature-login
git merge origin/main
```

📂 **GitHub Version Control Summary**

	and/Action
	ı rıe <url>
	a ckout -b branch-nı
	it (. + git commit -m
	) Ch origin branch-na
	PliitHub UI
	PliitHub UI or git me
	da origin main
	c lly fix, then add +

🧠 **Pro Tips**

- Use **.gitignore** to avoid pushing unwanted files (like node_modules/, *.log, etc.)
- Name branches clearly: feature/login, bug/fix-login, hotfix/typo
- Always **pull latest main** before creating a new branch
- Never commit directly to main on a team project

🧪 Hands-on Practice: S3 Bucket/Object Permissions

✅ Prerequisites

- AWS account
- S3 access via AWS Console or CLI
- IAM user with S3 permissions (AmazonS3FullAccess for now)

📂 Step 1: Create a New S3 Bucket

🖥 Using AWS Console:

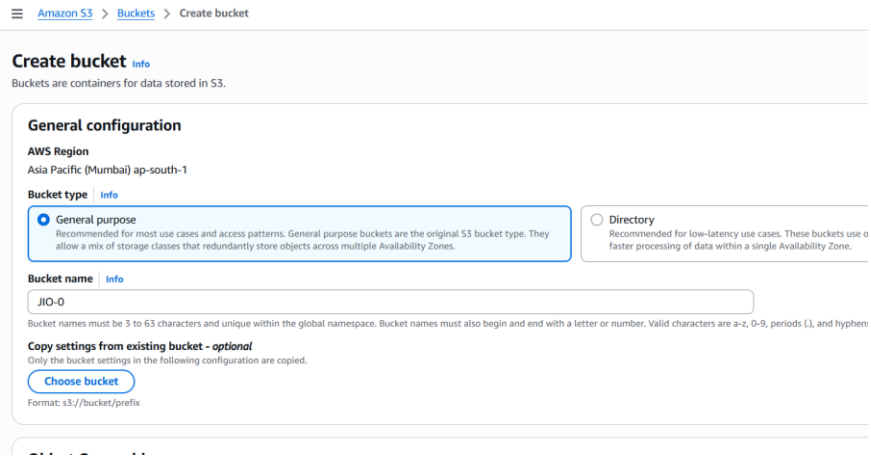
1. Go to S3 → Create Bucket
2. Bucket name: s3-permission-practice-bucket-<yourname>
3. Region: Select any
4. Uncheck: "Block all public access" (for practice only!)
5. Leave rest as default and create bucket

📄 Step 2: Upload an Object

- Open your bucket → Upload a file (e.g., hello.txt)
- After upload, click on the object → Note the Object URL

Right now, this object is private.

Try accessing it via browser — you'll get 403 Forbidden.



Object Ownership Info

Control ownership of objects written to this bucket from other AWS accounts and the use of access control lists (ACLs). Object ownership determines who can specify access to objects.

- (●) **ACLs disabled (recommended)**
  All objects in this bucket are owned by this account. Access to this bucket and its objects is specified using only policies.
- ( ) **ACLs enabled**
  Objects in this bucket can be owned by other AWS accounts. Access to this bucket and its objects can be specified using ACLs.

Object Ownership
Bucket owner enforced

Block Public Access settings for this bucket

Public access is granted to buckets and objects through access control lists (ACLs), bucket policies, access point policies, or all. In order to ensure that public access to this bucket and its objects is blocked, turn on Block all public access. These settings apply only to this bucket and its access points. AWS recommends that you turn on Block all public access, but before applying any of these settings, ensure that your applications will work correctly without public access. If you require some level of public access to this bucket or objects within, you can customize the individual settings below to suit your specific storage use cases. Learn more

- [ ] **Block *all* public access**
  Turning this setting on is the same as turning on all four settings below. Each of the following settings are independent of one another.
  - [ ] **Block public access to buckets and objects granted through *new* access control lists (ACLs)**
    S3 will block public access permissions applied to newly added buckets or objects, and prevent the creation of new public access ACLs for existing buckets and objects. This setting doesn't change any existing permissions that allow public access to S3 resources using ACLs.
  - [ ] **Block public access to buckets and objects granted through *any* access control lists (ACLs)**
    S3 will ignore all ACLs that grant public access to buckets and objects.
  - [ ] **Block public access to buckets and objects granted through *new* public bucket or access point policies**

Amazon S3 > Buckets

Successfully created bucket "jio-0"
To upload files and folders, or to configure additional bucket settings, choose **View details**.

View details

General purpose buckets | All AWS Regions | Directory buckets

General purpose buckets (1) Info

Buckets are containers for data stored in S3.

Copy ARN | Empty | Delete | Create bucket

Find buckets by name

Name	AWS Region	Creation date
jio-0	Asia Pacific (Mumbai) ap-south-1	August 5, 2025, 22:47:08 (UTC+05:30)

Account snapshot Info

View dashboard

Updated daily

Storage Lens provides visibility into storage usage and activity trends.

External access summary - *new* Info

Updated daily

External access findings help you identify bucket permissions that allow public access or access from other AWS accounts.

Amazon S3 > Buckets > jio-0 > Upload

Files and folders (2 total, 212.0 KB)

All files and folders in this table will be uploaded.

Remove | Add files | Add folder

Find by name

Name	Folder	Type	Size
txt.docx	-	application/vnd.openxmlformats-officedocume...	16.6 KB
Lord Ganesha in a different activity.png	-	image/png	195.4 KB

Destination Info

Destination

s3://jio-0

Destination details

Bucket settings that impact new objects stored in the specified destination.

Upload succeeded

For more information, see the **Files and folders** table.

After you navigate away from this page, the following information is no longer available.

Summary

Destination	Succeeded	Failed
s3://jio-0	2 files, 212.0 KB (100.00%)	0 files, 0 B (0%)

Files and folders | Configuration

Files and folders (2 total, 212.0 KB)

Find by name

Name	Folder	Type	Size	Status	Error
txt.docx	-	application/vnd.openxmlform...	16.6 KB	Succeeded	-
Lord Ganesha in a different activ...	-	image/png	195.4 KB	Succeeded	-

Amazon S3 > Buckets > jio-0

jio-0 Info

Objects | Properties | Permissions | Metrics | Management | Access Points

Objects

Copy S3 URI | Copy URL | Download | Open | Delete | Actions | Create folder | Upload

Objects are the fundamental entities stored in Amazon S3. You can use Amazon S3 inventory to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. Learn more

Find objects by prefix

Name	Type	Last modified	Size	Storage class
Lord Ganesha in a different activity.png	png	August 5, 2025, 22:50:38 (UTC+05:30)	195.4 KB	Standard
txt.docx	docx	August 5, 2025, 22:50:36 (UTC+05:30)	16.6 KB	Standard

Docker AWS S3 Guide | Lord Ganesha in a different acti | jio-0.s3.ap-south-1.amazonaws.

https://jio-0.s3.ap-south-1.amazonaws.com/Lord+Ganesha+in+a+different+activity.png

Just a moment... | DeepSeek - Into the... | Console Home | Co... | Console Home | ap-... | AWS Certified Cloud... | Your learning plan -... | Trailhead | The fun...

This XML file does not appear to have any style information associated with it. The document tree is shown below.


```
▼<Error>
   <Code>AccessDenied</Code>
   <Message>Access Denied</Message>
   <RequestId>6KE09ZTZQNZ94KJH</RequestId>
   <HostId>6Vm6AgzVqj5Y8V+Br5KBaus3BexjFeJPg1ntqKJ0pE+0kYCHUd0X7IvgFZqTC4QK1AYMvp2mx9QDTX4UqdpMrjrbiJdq9pWY</HostId>
 </Error>
```


Edit Object Ownership Info

Object Ownership

Control ownership of objects written to this bucket from other AWS accounts and the use of access control lists (ACLs). Object ownership determines who can specify access to objects.

- ○ **ACLs disabled (recommended)**
  All objects in this bucket are owned by this account. Access to this bucket and its objects is specified using only policies.
- ◉ **ACLs enabled**
  Objects in this bucket can be owned by other AWS accounts. Access to this bucket and its objects can be specified using ACLs.

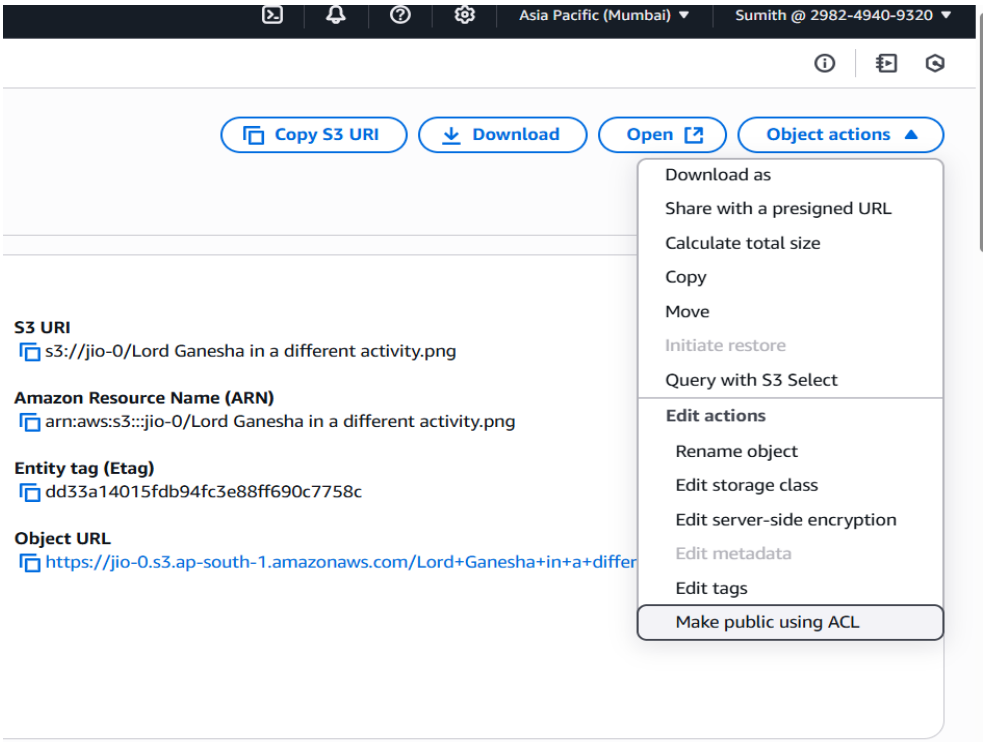
⚠ We recommend disabling ACLs, unless you need to control access for each object individually or to have the object writer own the data they upload. Using a bucket policy instead of ACLs to share data with users outside of your account simplifies permissions management and auditing.

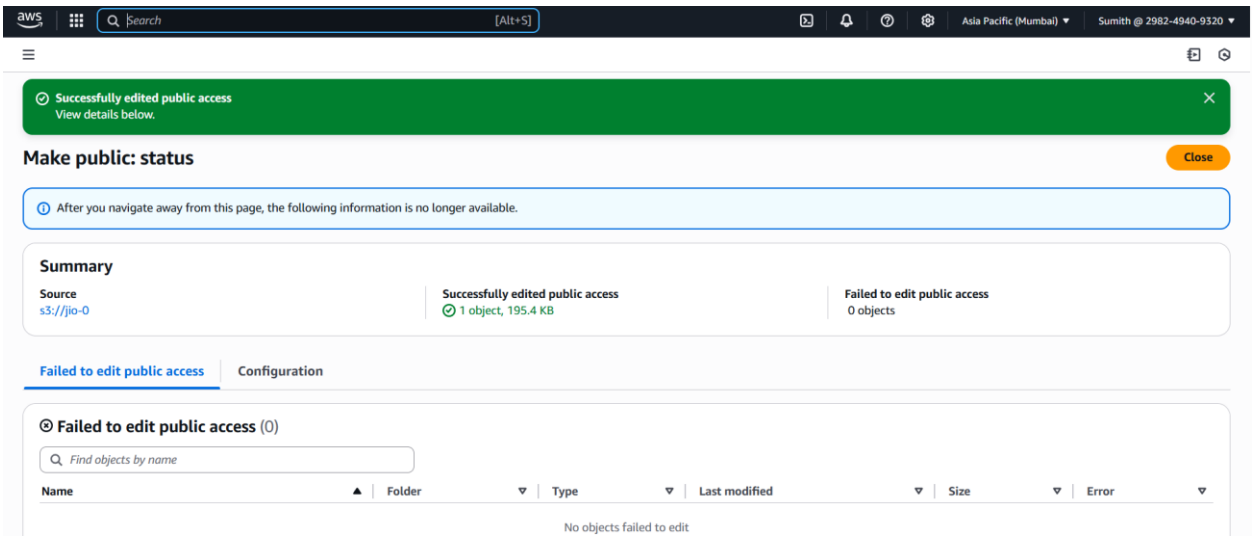
⚠ **Enabling ACLs turns off the bucket owner enforced setting for Object Ownership**
Once the bucket owner enforced setting is turned off, access control lists (ACLs) and their associated permissions are restored. Access to objects that you do not own will be based on ACLs and not the bucket policy.

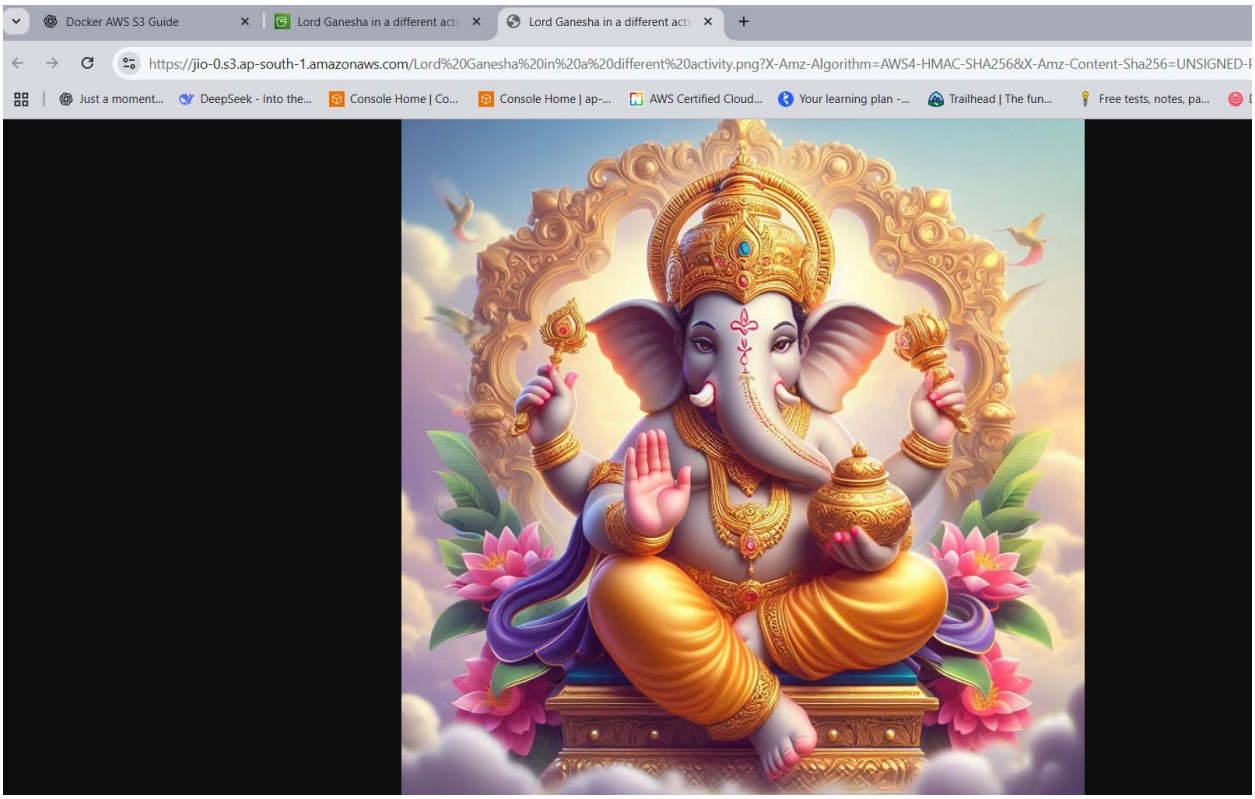
☐ I acknowledge that ACLs will be restored.

Object Ownership

- ◉ Bucket owner preferred
  If new objects written to this bucket specify the bucket-owner-full-control canned ACL, they are owned by the bucket owner. Otherwise, they are owned by the object writer.
- ○ Object writer
  The object writer remains the object owner.


Asia Pacific (Mumbai)
Sumith @ 2982-4940-9320
Copy S3 URI
Download
Open
Object actions
Download as
Share with a presigned URL
Calculate total size
Copy
Move
Initiate restore
Query with S3 Select
Edit actions
Rename object
Edit storage class
Edit server-side encryption
Edit metadata
Edit tags
Make public using ACL
S3 URI
s3://jio-0/Lord Ganesha in a different activity.png
Amazon Resource Name (ARN)
arn:aws:s3:::jio-0/Lord Ganesha in a different activity.png
Entity tag (Etag)
dd33a14015fdb94fc3e88ff690c7758c
Object URL
https://jio-0.s3.ap-south-1.amazonaws.com/Lord+Ganesha+in+a+differ


Search
[Alt+S]
Asia Pacific (Mumbai)
Sumith @ 2982-4940-9320
Successfully edited public access
View details below.
Make public: status
Close
After you navigate away from this page, the following information is no longer available.
Summary
Source
s3://jio-0
Successfully edited public access
1 object, 195.4 KB
Failed to edit public access
0 objects
Failed to edit public access
Configuration
Failed to edit public access (0)
Find objects by name
Name
Folder
Type
Last modified
Size
Error
No objects failed to edit


Docker AWS S3 Guide
Lord Ganesha in a different acti
Lord Ganesha in a different acti
https://jio-0.s3.ap-south-1.amazonaws.com/Lord%20Ganesha%20in%20a%20different%20activity.png?X-Amz-Algorithm=AWS4-HMAC-SHA256&X-Amz-Content-Sha256=UNSIGNED-
Just a moment...
DeepSeek - Into the...
Console Home | Co...
Console Home | ap-...
AWS Certified Cloud...
Your learning plan -...
Trailhead | The fun...
Free tests, notes, pa...